

AniMood

Complete Backend Build Specification — Handoff for Claude Code

This is the handoff specification for AniMood's backend and frontend infrastructure. It is written so Claude Code can execute it inside a GitHub repo, using your Railway Pro subscription, OpenAI API credits, and Reddit/MyAnimeList API access.

The system: a legally-safe emotional anime/manga/manhwa discovery platform. n8n workflows ingest public fandom discussion signals, AI extracts validated emotional patterns, you approve them in a review dashboard, approved data flows to a Next.js public site at **animood.app**. **Hard rule everywhere:** no piracy hosting, no scraped content republished, no AI-generated emotional labels without evidence.

Confirmed assets: Railway Pro • animood.app domain • OpenAI API key with credits • GitHub PAT • Claude Code.

Confirmed stack: GPT-5.4 mini (extraction) + GPT-5.4 nano (cheap ops) • Next.js 15 + Tailwind + shadcn/ui + Drizzle ORM • Postgres 16 + pgvector • n8n self-hosted on Railway • Reddit + MAL APIs.

Operating mode: Phase A (manual triggers, user-only review) → Phase B (scheduled) after exit criteria pass.

Contents

#	Section
0	How to Use This Document with Claude Code
1	System Architecture
2	Cost Estimate & Two-Phase Plan
3	Railway Setup — Services to Provision
4	Environment Variables & API Keys
5	GitHub Repository Structure
6	Database Schema (Postgres + pgvector)
7	n8n Workflows (W1–W9)
8	AI Prompt Templates
9	Review Dashboard Spec
10	Next.js Frontend — Structure & Routes
11	SEO Architecture
12	Claude Code Execution Plan — Commit-by-Commit
13	Manual User Tasks (only you can do)
14	First-Day Run-Through
15	Phase A → Phase B Validation Criteria

#	Section
16	Failure Modes & Debugging Guide
17	Parallel SEO Learning Track

0. How to Use This Document with Claude Code

This document is structured as a single source of truth. To work with it correctly:

- Read sections 1–4 yourself first. They are decisions you must understand, not delegate.
- Sections 5–11 are the build spec. Claude Code reads these and executes.
- Section 12 is the commit-by-commit plan. Give Claude Code ONE commit task per turn.
- Section 13 lists tasks only you can do (sign up for APIs, etc.).
- Section 14 is the first-day script. Follow line by line.
- Section 15 is the exit gate from validation to production.

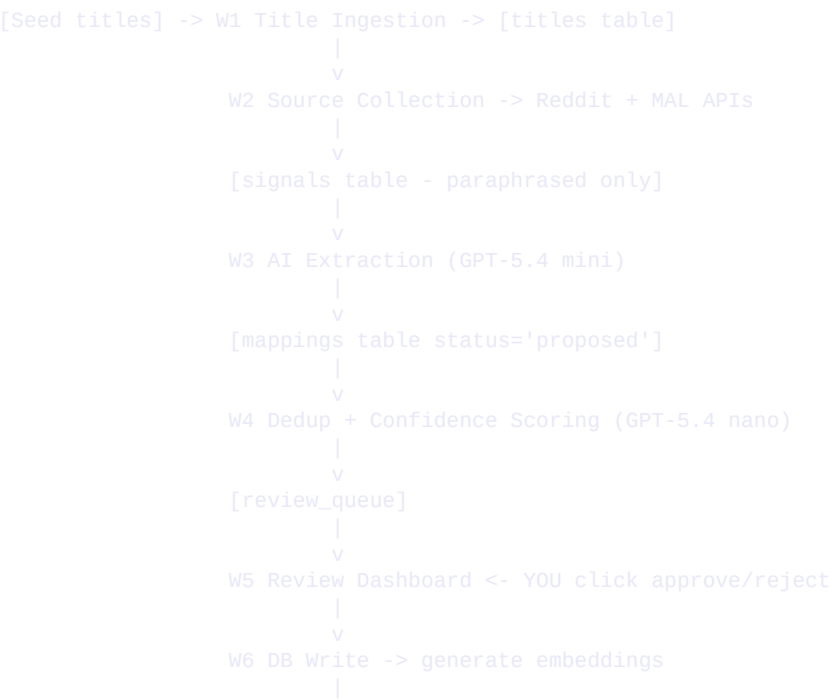
Critical execution rule: Format every prompt to Claude Code as: 'Read sections X–Y of the spec. Implement only commit task N. Stop when done and tell me what to verify.' Do NOT paste the entire document and say 'build everything.' Claude Code is excellent at scoped tasks and gets confused with unscoped ones.

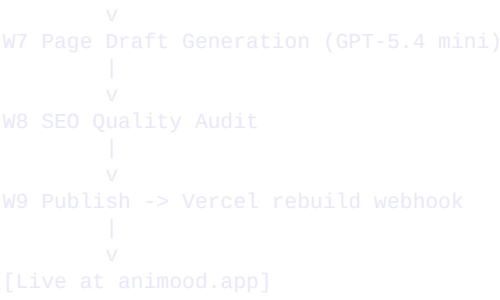
1. System Architecture

AniMood has six components. Five hosted on Railway, the public site on Vercel for SEO speed.

Component	Purpose	Tech	Host
Postgres DB	All entities + vector embeddings	Postgres 16 + pgvector	Railway
n8n	9 data pipelines	n8n Docker	Railway
Review Dashboard API	Backend for approve/reject UI	Node.js + Express + Drizzle	Railway
Review Dashboard UI	Where YOU approve mappings	Next.js	Railway
Public site	animood.app	Next.js 15 + Tailwind + shadcn	Vercel free
External APIs	Data + AI calls	REST/GraphQL	External

1.1 Data flow





2. Cost Estimate & Two-Phase Plan

Phase A is cheap. Phase B scales with frequency. Stay in Phase A until exit criteria (Section 15).

2.1 Phase A — Validation (~4 weeks)

Service	Approx Cost
Railway Pro (you already pay)	■0 incremental
Postgres + n8n + Dashboard on Railway	Within Pro credit
OpenAI API (low volume)	■100–300/mo
Reddit + MAL APIs	Free
Vercel Hobby	Free
TOTAL Phase A	~■100–500/mo

2.2 Phase B — Production

Schedule	OpenAI cost	Total monthly
Daily extraction	■800–2,500/mo	■2,000–3,700
Weekly extraction	■200–600/mo	■1,400–1,800
On-demand only	■50–200/mo	■1,250–1,400

Recommendation: Phase A = manual triggers only. Phase B start = weekly schedule. Daily only if traffic justifies it. Most successful content sites run weekly. Cost spikes kill projects; protect your runway.

3. Railway Setup — Services to Provision

Create one Railway project called **animood**. Provision 4 services inside it.

3.1 Service: postgres

- Railway dashboard → New Service → Database → PostgreSQL
- Wait ~1 min for provisioning
- Variables tab → copy DATABASE_URL
- Data tab → run: CREATE EXTENSION IF NOT EXISTS vector;
- Verify: SELECT extversion FROM pg_extension WHERE extname = 'vector';

3.2 Service: n8n

```
Type: Docker Image
Image: n8nio/n8n:latest
Port: 5678
```

```
Environment variables:
N8N_BASIC_AUTH_ACTIVE=true
N8N_BASIC_AUTH_USER=admin
N8N_BASIC_AUTH_PASSWORD=<strong-password>
N8N_HOST=animood-n8n.up.railway.app
N8N_PROTOCOL=https
N8N_PORT=5678
WEBHOOK_URL=https://animood-n8n.up.railway.app/
GENERIC_TIMEZONE=Asia/Kolkata
DB_TYPE=postgresdb
```

```
DB_POSTGRESDB_HOST=${Postgres.PGHOST}
DB_POSTGRESDB_PORT=${Postgres.PGPORT}
DB_POSTGRESDB_DATABASE=${Postgres.PGDATABASE}
DB_POSTGRESDB_USER=${Postgres.PGUSER}
DB_POSTGRESDB_PASSWORD=${Postgres.PGPASSWORD}
```

Settings -> Networking -> Generate Domain

3.3 Services: dashboard-api + dashboard-ui

- Both deployed from the GitHub repo (Section 5)
- Connect Railway to GitHub, point at apps/api and apps/dashboard folders
- Auto-deploys on push to main branch
- Env vars set later (Section 4)

4. Environment Variables & API Keys

Master .env template (root of repo):

```
# --- Database ---
DATABASE_URL=postgresql://user:pass@host:port/dbname

# --- OpenAI ---
OPENAI_API_KEY=sk-...
OPENAI_MODEL_EXTRACTION=gpt-5.4-mini
OPENAI_MODEL_CHEAP=gpt-5.4-nano

# --- Reddit API (reddit.com/prefs/apps, create 'script' app) ---
REDDIT_CLIENT_ID=...
REDDIT_CLIENT_SECRET=...
REDDIT_USER_AGENT=AniMoodBot/1.0 by /u/yourusername
REDDIT_USERNAME=...
REDDIT_PASSWORD=...

# --- MyAnimeList (myanimelist.net/apiconfig) ---
MAL_CLIENT_ID=...

# --- n8n ---
N8N_BASE_URL=https://animood-n8n.up.railway.app
N8N_API_KEY=...

# --- Dashboard ---
DASHBOARD_ADMIN_EMAIL=you@example.com
DASHBOARD_ADMIN_PASSWORD_HASH=<bcrypt>
SESSION_SECRET=<random 32 bytes>

# --- Public site ---
NEXT_PUBLIC_SITE_URL=https://animood.app
NEXT_PUBLIC_BASE_DOMAIN=animood.app
REVALIDATE_SECRET=<random>
VERCEL_DEPLOY_HOOK_URL=https://api.vercel.com/v1/integrations/deploy/...
```

Keys you must manually obtain (Section 13 has the order):

Service	Where	Time
OpenAI API key	platform.openai.com/api-keys	2 min
Reddit credentials	reddit.com/prefs/apps -> 'script' type	5 min
MAL Client ID	myanimelist.net/apiconfig	5 min
Railway API token	railway.app -> Account -> Tokens	1 min
GitHub PAT	github.com -> Developer settings -> Tokens	2 min
Vercel Deploy Hook	vercel.com -> Project -> Settings -> Git	1 min

5. GitHub Repository Structure

Single monorepo. Claude Code creates this on first commit.

```
animood/
├── README.md
├── .env.example
├── .gitignore
├── package.json                # workspace root
├── pnpm-workspace.yaml
├── turbo.json
├──
├── apps/
│   ├── web/                   # Public site (animood.app)
│   │   ├── app/              # Next.js 15 App Router
│   │   │   ├── (marketing)/
│   │   │   ├── emotion/[slug]/
│   │   │   ├── anime/[slug]/
│   │   │   ├── manga/[slug]/
│   │   │   ├── manhwa/[slug]/
│   │   │   ├── character/[slug]/
│   │   │   ├── life-stage/[slug]/
│   │   │   ├── theme/[slug]/
│   │   │   ├── compare/[slug]/
│   │   │   ├── debate/[slug]/
│   │   │   ├── taste-profile/[type]/
│   │   │   ├── quiz/
│   │   │   ├── where-to-watch/[slug]/
│   │   │   ├── sitemap.xml/route.ts
│   │   │   ├── robots.txt/route.ts
│   │   │   └── api/revalidate/route.ts
│   │   └──
│   ├── dashboard/           # Review dashboard UI (Next.js)
│   └── api/                  # Dashboard API + n8n webhook receiver
│   └──
├── packages/
│   ├── db/                   # Drizzle schema + migrations
│   │   ├── schema/
│   │   ├── migrations/
│   │   ├── seeds/
│   │   └── client.ts
│   ├── ai/                   # OpenAI wrapper + prompts (md files)
│   │   ├── prompts/
│   ├── seo/                  # schema.org + sitemap utils
│   └── ui/                   # Shared shadcn components
│   └──
├── n8n-workflows/            # 9 workflow JSON exports for version control
│   ├── W1-title-ingestion.json
│   ├── W2-source-collection.json
│   ├── W3-ai-extraction.json
│   ├── W4-dedup-score.json
│   ├── W5-review-queue.json
│   ├── W6-db-write.json
│   ├── W7-page-draft-gen.json
│   ├── W8-seo-audit.json
│   └── W9-publish.json
│   └──
├── docs/
│   ├── ARCHITECTURE.md
│   ├── WORKFLOWS.md
│   ├── DATABASE.md
│   └── PROMPTS.md
│   └──
├── scripts/
│   ├── setup.sh
│   ├── seed-database.ts
│   └── deploy.sh
```

6. Database Schema (Postgres + pgvector)

Drizzle ORM is the source of truth. Schema in packages/db/schema/. Migrations auto-generated.

6.1 Tables overview

Table	Purpose
titles	Anime/manga/manhwa entries
emotions	Controlled vocab of emotional states
life_stages	Life-phase categories
themes	Narrative/philosophical themes
character_psychologies	Psychology categories
characters	Specific characters
mappings	The core graph (title↔emotion, etc.)
signals	Paraphrased extraction snippets
sources	Origin metadata only — no raw content
page_drafts	Generated pages awaiting review
pages	Published live pages
user_profiles	Anonymous taste profiles
votes	User match votes
embeddings	Vector embeddings (pgvector)
workflow_logs	Run history for all 9 workflows
review_queue	Mappings/drafts waiting for you

6.2 Critical schema: titles

```
// packages/db/schema/titles.ts
import { pgTable, serial, text, integer, timestamp, pgEnum } from 'drizzle-orm/pg-core';

export const titleType = pgEnum('title_type',
  ['anime', 'manga', 'manhwa', 'manhua', 'light_novel']);
export const titleStatus = pgEnum('title_status',
  ['completed', 'ongoing', 'hiatus', 'cancelled', 'announced']);

export const titles = pgTable('titles', {
  id: serial('id').primaryKey(),
  slug: text('slug').notNull().unique(),
  name: text('name').notNull(),
  nameOriginal: text('name_original'),
  type: titleType('type').notNull(),
  status: titleStatus('status').notNull(),
  releaseYear: integer('release_year'),
  endYear: integer('end_year'),
  demographic: text('demographic'),
  description: text('description'),
  spoilerSafeSummary: text('spoiler_safe_summary'),
  emotionalPositioning: text('emotional_positioning'),
  malId: integer('mal_id').unique(),
  anilistId: integer('anilist_id').unique(),
  lastSignalCollectedAt: timestamp('last_signal_collected_at'),
  createdAt: timestamp('created_at').defaultNow(),
  updatedAt: timestamp('updated_at').defaultNow(),
});
```


6.3 Critical schema: mappings (the core graph)

```
export const mappingType = pgEnum('mapping_type', [
  'title_emotion', 'title_theme', 'title_life_stage',
  'character_psychology', 'title_similar_to', 'character_similar_to',
  'opposes_interpretation', 'softer_alternative_to', 'intensity_upgrade_from'
]);
export const confidence = pgEnum('confidence_level',
  ['low', 'medium', 'high', 'verified']);
export const mappingStatus = pgEnum('mapping_status', [
  'proposed', 'evidence_collected', 'human_reviewed', 'published',
  'contested', 'retired'
]);

export const mappings = pgTable('mappings', {
  id: serial('id').primaryKey(),
  type: mappingType('type').notNull(),
  sourceTable: text('source_table').notNull(), // 'titles' or 'characters'
  sourceId: integer('source_id').notNull(),
  targetTable: text('target_table').notNull(),
  targetId: integer('target_id').notNull(),
  intensity: integer('intensity'), // 1-5
  evidenceNotes: text('evidence_notes').notNull(),
  evidenceCount: integer('evidence_count').default(1),
  confidence: confidence('confidence').notNull(),
  confidenceScore: decimal('confidence_score', { precision:4, scale:3 }),
  status: mappingStatus('status').notNull().default('proposed'),
  reviewedBy: text('reviewed_by'),
  reviewedAt: timestamp('reviewed_at'),
  createdAt: timestamp('created_at').defaultNow(),
});
```

6.4 Critical schema: embeddings (semantic search)

```
// pgvector for similarity search later
export const embeddings = pgTable('embeddings', {
  id: serial('id').primaryKey(),
  entityTable: text('entity_table').notNull(),
  entityId: integer('entity_id').notNull(),
  textSource: text('text_source').notNull(),
  embedding: vector('embedding', { dimensions: 1536 }),
  createdAt: timestamp('created_at').defaultNow(),
});

// After migration, run manually:
// CREATE INDEX ON embeddings USING ivfflat (embedding vector_cosine_ops) WITH (lists=100);
```

6.5 Critical schema: signals (NEVER raw text)

```
// CRITICAL: extractedPattern must be paraphrased by AI, never the original text.
// Blueprint Section 12 – no raw scraping displayed publicly.

export const signals = pgTable('signals', {
  id: serial('id').primaryKey(),
  titleId: integer('title_id').references(() => titles.id),
  characterId: integer('character_id').references(() => characters.id),
  sourceType: sourceType('source_type').notNull(),
  sourceUrl: text('source_url'), // reference only
  sourceDate: timestamp('source_date'),
  extractedPattern: text('extracted_pattern').notNull(), // PARAPHRASED
  detectedEmotionId: integer('detected_emotion_id'),
  confidenceScore: integer('confidence_score'),
  reviewedStatus: text('reviewed_status').default('pending'),
  createdAt: timestamp('created_at').defaultNow(),
});
```

7. n8n Workflows (W1–W9)

Each workflow exported as JSON in n8n-workflows/ for version control. Configurations below.

W1 — Title Ingestion

Node	Type	Config
Trigger	Manual (A) / Schedule (B)	Cron: Sun 02:00 IST in B
Read Seed	Google Sheets / DB query	Sheet ID env var
Normalize	Function (JS)	Slugify, lowercase, dedupe vs existing
MAL Lookup	HTTP Request	GET api.myanimelist.net/v2/anime?q={{name}}
Anilist Lookup	HTTP Request	GraphQL POST anilist.co/graphql
Merge	Merge node	Combine MAL + Anilist data
Insert	Postgres	INSERT ... ON CONFLICT (slug) DO UPDATE
Log	Postgres	INSERT INTO workflow_logs

W2 — Source Collection

Node	Type	Config
Trigger	Manual / Webhook from W1	
Get Pending Titles	Postgres	WHERE last_signal_collected_at < NOW() - INTERVAL '7 days'
Split Batches	Split In Batches	5 per batch
Reddit Search	HTTP Request	Reddit API search by title name
MAL Reviews	HTTP Request	GET api/v2/anime/{{mal_id}}/reviews
Filter Quality	Function	Drop short / low-engagement items
Paraphrase	OpenAI (5.4 nano)	Prompt 8.1 — paraphrase, never quote
Insert Signals	Postgres	INSERT INTO signals
Update Title	Postgres	SET last_signal_collected_at = NOW()

W3 — AI Emotional Extraction

Node	Type	Config
Trigger	Webhook from W2 / Manual	
Get Unprocessed	Postgres	WHERE reviewed_status = 'pending' LIMIT 50
Group by Title	Function	Cluster signals per title_id
Extract	OpenAI (5.4 mini)	Prompt 8.2 — Emotional Extraction
Parse JSON	Function	Validate, reject malformed
Insert Candidates	Postgres	INSERT INTO mappings status='proposed'
Mark Processed	Postgres	UPDATE signals SET reviewed_status='extracted'

W4 — Dedup + Confidence Scoring

Node	Type	Config
Trigger	After W3 / Manual	
Get Recent	Postgres	status='proposed' AND created > NOW() - INTERVAL '1 day'
Detect Dupes	OpenAI (5.4 nano)	Cluster similar emotion claims per title
Score	Function	evidence_count + source diversity → confidence_score
Update	Postgres	SET confidence_score, status='evidence_collected'
Queue Review	Postgres	INSERT INTO review_queue

W5 — Review Queue (integration with dashboard)

Node	Type	Config
Webhook	Webhook	POST /webhook/review-decision
Validate	Function	Check mapping_id + decision
Route	Switch	approve→W6, reject→retire, edit→update+W6
Update	Postgres	SET status='human_reviewed', reviewedBy, reviewedAt
Trigger W6	HTTP Request	POST mapping_id to W6 webhook

W6 — Database Write

Node	Type	Config
Webhook	From W5	
Get Mapping	Postgres	SELECT WHERE id={{mapping_id}}
Embed	OpenAI Embeddings	model: text-embedding-3-small
Insert Embedding	Postgres	INSERT INTO embeddings
Update Status	Postgres	SET status='published'
Trigger W7	HTTP Request	If new pairing, generate page

W7 — Page Draft Generation

Node	Type	Config
Trigger	Webhook / Manual	
Identify Page Type	Function	emotion / title / character / etc.
Gather Data	Postgres	All mappings + related entities
Generate Draft	OpenAI (5.4 mini)	Prompt 8.3 — Page Generation
Inject Schema	Function	Build schema.org JSON-LD
Add Internal Links	Function	Min 5 internal links rule
Insert Draft	Postgres	INSERT INTO page_drafts status='pending_review'

W8 — SEO Quality Audit

Node	Type	Config
Trigger	After W7	

Node	Type	Config
Get Pending Draft	Postgres	status='pending_review'
Word Count Check	Function	400-2500 words
Links Check	Function	5+ internal, 2+ outbound official
Schema Validation	Function	Valid JSON-LD
Copyright Check	OpenAI nano	Detect quoted text > 15 words
Generic Check	OpenAI nano	Flag AI-fluff filler
Pass or Flag	Postgres	review_passed or review_flagged

W9 — Publish Trigger

Node	Type	Config
Webhook	From dashboard approve	
Move Draft to Pages	Postgres	INSERT INTO pages FROM page_drafts
Update Sitemap	Function	Add URL to sitemap pending list
Trigger Rebuild	HTTP Request	POST Vercel Deploy Hook
Log	Postgres	INSERT INTO workflow_logs
Notify You	Email/Telegram (optional)	'Page live at animood.app/...'

8. AI Prompt Templates

Store full prompts in packages/ai/prompts/ as Markdown files. n8n nodes load them by filename.

8.1 Prompt: Paraphrase Signal (W2, GPT-5.4 nano)

```
SYSTEM:
You are a paraphrasing assistant. You receive a public comment about an anime/manga/manhwa.
Extract the EMOTIONAL or THEMATIC PATTERN the commenter expressed, in your own words.
Never quote the original text. Never reproduce > 5 consecutive words verbatim.

Return JSON only:
{
  "pattern": "<one-sentence paraphrase>",
  "emotion_signals": ["loneliness", "rebuilding"],
  "intensity_hint": 1-5 or null,
  "confidence_in_signal": 0-100
}

USER:
Title: {{title.name}}
Source: {{source.type}} ({{source.url}})
Original text:
""{{source.text}}""

Return ONLY the JSON object. No preamble.
```

8.2 Prompt: Emotional Extraction (W3, GPT-5.4 mini)

```
SYSTEM:
You are an emotional analysis system for AniMood.
Extract validated emotional relationships from fan discussion patterns.

NON-NEGOTIABLE RULES:
1. Only return emotions backed by 2+ independent signals OR clear narrative evidence.
2. Never invent emotions. If signals are weak, return empty array.
3. Every emotion must be from the AniMood ontology (provided).
4. Evidence Notes must be in your own words, reference specific story elements,
   never quote sources.
5. Confidence: 'high' = 3+ signals or clear narrative. 'medium' = 2 signals.
   'low' otherwise.

Emotion Ontology:
[Loneliness, Healing, Revenge, Redemption, Ambition, Grief, Hope,
Existential dread, Peaceful comfort, Emotional devastation, Identity crisis,
Moral ambiguity, Burnout, Rebuilding, Nostalgia, Freedom]

Return JSON:
{
  "title_slug": "...",
  "extracted_mappings": [
    {
      "emotion": "Loneliness",
      "intensity": 4,
      "evidence_notes": "...",
      "supporting_signal_count": 5,
      "confidence": "high"
    }
  ]
}

USER:
Title: {{title.name}} ({{title.type}}, {{title.year}})
Signal patterns ({{signal_count}}):
{{#each signals}}
- ({{this.source_type}}) ({{this.pattern}})
{{/each}}

Return only the JSON object.
```

8.3 Prompt: Page Draft Generation (W7, GPT-5.4 mini)

```
SYSTEM:
```

```
You generate page content for AniMood, a legally-safe emotional discovery platform.

RULES:
1. Use ONLY data provided. Do not invent facts.
2. Never quote external sources. No copyrighted text.
3. Tone: emotionally intelligent but grounded. No fake therapy language.
   Use phrasings like "many viewers connect this with" or "this may resonate if".
4. Include exactly the sections specified in the template.
5. Each section must contain 2+ internal link placeholders {{link:type:slug}}.
6. Word count: emotion pages 600-1200, title pages 800-1500.

USER:
Generate a {{page_type}} page for: {{entity.name}}
Template: {{template_name}}
Validated data:
{{data_json}}

Return Markdown content only. Use {{link:type:slug}} syntax for internal links.
```

9. Review Dashboard Spec

Minimal Next.js app. Email+password auth. Only you log in. Built for SPEED of review.

9.1 Routes

Route	Purpose
/login	Email + password
/dashboard	Counts: pending mappings, drafts, recent activity
/dashboard/mappings	Queue: Approve / Reject / Edit
/dashboard/drafts	Generated page drafts awaiting review
/dashboard/mapping/[id]	Detail: signals, evidence, AI confidence
/dashboard/draft/[id]	Page preview. Approve to publish
/dashboard/workflows	Trigger workflows manually, run history
/dashboard/settings	Toggle Phase A ↔ Phase B

9.2 Mapping review screen (the only one that matters)

- Title + emotion + intensity (all shown at top)
- Evidence notes from AI
- Source breakdown: 5 Reddit + 2 MAL + 1 Anilist (per-source confidence)
- Three buttons: **Approve / Reject / Edit**
- Keyboard shortcuts: A approve, R reject, E edit, J/K navigate
- Target: 10 seconds per mapping

Speed matters. If reviewing feels slow, you'll quit. The whole pipeline depends on this dashboard being fast. Test with 50 sample mappings before going live.

10. Next.js Frontend — Structure & Routes

The public site at animood.app. Next.js 15 App Router for SSG/ISR. Vercel for edge caching. Reads Postgres via Drizzle. Revalidates on W9 webhook.

URL Pattern	Page Type	Source
/	Homepage with mood cards	Static
/emotion/[slug]	Emotion page	Generated by W7
/anime/[slug]	Anime title page	W7
/manga/[slug]	Manga title page	W7
/manhwa/[slug]	Manhwa title page	W7
/character/[slug]	Character profile	W7
/life-stage/[slug]	Life-stage discovery	W7
/theme/[slug]	Theme exploration	W7
/compare/[slug]	Title vs title emotional comparison	W7
/debate/[slug]	Opinion spectrum	W7
/taste-profile/[type]	Quiz result page	Templated
/quiz	Taste profile quiz	Static + client state
/where-to-watch/[slug]	Official source guide	W7
/sitemap.xml	Dynamic sitemap	Runtime
/robots.txt	Robots file	Runtime

Rendering strategy:

- **Static at build:** homepage, /quiz, /taste-profile/[type] templates
- **ISR (Incremental Static Regeneration):** all entity pages, on-demand via /api/revalidate
- **Edge runtime:** /sitemap.xml, /robots.txt
- **Client-only:** quiz state, vote interactions (localStorage initially)

11. SEO Architecture

Implements modern entity-based SEO. The frontend wires these patterns.

11.1 Schema.org JSON-LD on every entity page

```
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "TVSeries",
  "name": "Vinland Saga",
  "alternateName": "■■■■■■■■■■",
  "genre": ["Seinen", "Historical", "Drama"],
  "datePublished": "2019-07-08",
  "description": "...",
  "character": [
    { "@type": "Person", "name": "Thorfinn",
      "url": "https://animood.app/character/thorfinn" }
  ],
  "isRelatedTo": [
    { "@type": "TVSeries", "name": "Vagabond",
```

```
    "url": "https://animood.app/manga/vagabond"}
  ],
  "about": [
    { "@type": "Thing", "name": "Redemption",
      "url": "https://animood.app/theme/redemption" }
  ]
}
```

</script>

11.2 Internal linking rule (enforced in W8)

Every entity page links to:

- 2+ related emotion pages
- 3+ related title pages
- 1+ life-stage page
- 1+ character page (where applicable)
- 1+ debate/opinion-spectrum page (where applicable)
- Anchor text always carries entity relationship — never 'click here'

11.3 Sitemap strategy

- /sitemap.xml is a sitemap index pointing to topic-specific sitemaps
- /sitemaps/emotions.xml, /sitemaps/titles.xml, /sitemaps/characters.xml, etc.
- Each sub-sitemap rebuilt on Vercel ISR when new entity is added
- Submitted to Google Search Console and Bing Webmaster Tools on day 1

12. Claude Code Execution Plan — Commit by Commit

The most important section. ONE commit task per Claude Code turn. Verify before moving on. This prevents Claude Code from creating a tangled mess across the whole repo.

#	Commit Task	Verify Before Next Step
1	Init monorepo: package.json, pnpm-workspace, turbo.json, tsconfig, README	pnpm install, README only
2	Create packages/db with Drizzle schema (all tables from 667) drizzle-kit generate makes clean migration	drizzle-kit generate makes clean migration
3	Add Postgres connection, run first migration against Railway DB	psql DB shows all tables + vector ext
4	Create packages/ai with prompt files + OpenAI wrapper	extract.ts returns valid JSON on test input
5	Create packages/seo with schema + sitemap utils	Unit tests pass
6	Build apps/api Express server with /health endpoint	Deployed to Railway, /health returns 200
7	Dashboard API endpoints (list/approve/reject/edit mappings)	API endpoints respond correctly via curl
8	Build apps/dashboard: login + mappings queue page	You can log in, queue page loads
9	Drafts queue, mapping detail, draft preview pages	Full review flow works with seed data
10	Seed DB with 40 titles from AniMood_Seed_Data.xlsx	SELECT COUNT(*) FROM titles = 40
11	Build apps/web skeleton: homepage with mood cards	Deploys to Vercel, loads at animood.app
12	Add /emotion/[slug] with ISR + schema.org	Sample page valid in Google Rich Results Test
13	Add /anime/[slug], /manga/[slug], /manhwa/[slug] routes	Three sample pages render
14	Add /character, /life-stage, /theme routes	Pages render with internal links
15	Add /quiz with taste profile logic	Quiz completes, shows result page
16	Add /compare/[slug] and /debate/[slug] routes	Sample pages render
17	Add /sitemap.xml + /robots.txt + revalidate API	Sitemap valid XML, robots correct
18	Set up n8n on Railway from §3.2 instructions	n8n login page loads
19	Import W1 (Title Ingestion) workflow	Manual trigger inserts 5 test titles
20	Import W2 (Source Collection) workflow	Reddit + MAL signals come in for 1 title
21	Import W3 (AI Extraction) workflow	5+ candidate mappings appear in DB
22	Import W4 (Dedup + Score) workflow	Mappings have confidence_score values
23	Import W5 (Review Queue webhook)	Approve in dashboard triggers W5→W6
24	Import W6 (DB Write) workflow	Approved mappings get embeddings + status='published'
25	Import W7 (Page Draft Gen)	Draft pages appear in dashboard
26	Import W8 (SEO Quality Audit)	Drafts get review_passed or review_flagged
27	Import W9 (Publish)	Approved draft triggers Vercel rebuild
28	End-to-end test: 1 new title → live page	Full pipeline completes for 1 title
29	Polish: error handling, logging, retries	All workflows have try/catch + log to workflow_logs
30	Documentation: README, deployment guide, runbook	Anyone could rebuild from docs alone

Verification rule: Don't proceed to commit N+1 until you've personally verified commit N works. This will feel slow. It's saving you from rebuilding the whole thing in 2 weeks when something fundamental breaks because commit 3 had a bug nobody caught.

13. Manual User Tasks

Things Claude Code can't do for you. Do these in order. Estimated total time: ~90 min.

#	Task	Where	Time
1	Create GitHub repo 'animood'	github.com/new	2 min
2	Generate GitHub PAT (repo + workflow scopes)	github.com/settings/tokens	2 min
3	Sign up for OpenAI API + add \$10 credits	platform.openai.com	10 min
4	Generate OpenAI API key	platform.openai.com/api-keys	1 min
5	Create Reddit script app	reddit.com/prefs/apps	5 min
6	Register MAL API client	myanimelist.net/apiconfig	5 min
7	Create new Railway project 'animood'	railway.app	2 min
8	Provision Postgres service (§3.1)	Railway dashboard	3 min
9	Enable vector extension in Postgres	Railway data tab	1 min
10	Provision n8n service (§3.2)	Railway dashboard	10 min
11	Sign up for Vercel + import GitHub repo	vercel.com	5 min
12	Add animood.app domain to Vercel + DNS setup	Vercel + your registrar	10 min
13	Create Vercel deploy hook	Vercel project settings	1 min
14	Generate session secret (random 32 bytes)	openssl rand -base64 32	1 min
15	Generate bcrypt hash of admin password	https://bcrypt.online (or local)	2 min
16	Add all environment variables to Railway	Each service's Variables tab	10 min
17	Add OPENAI_API_KEY + DATABASE_URL to Vercel	Vercel project env vars	3 min
18	Submit animood.app to Google Search Console	search.google.com/search-console	5 min
19	Submit to Bing Webmaster Tools	bing.com/webmasters	5 min

14. First-Day Run-Through

Step-by-step first run, ~6 hours total spread across 2 days.

Day 1 morning (3 hours):

- Manual tasks 1–8 from §13
- Give Claude Code commit task 1 from §12
- Verify, then commit 2
- Continue through commit 5
- **Checkpoint:** packages/db, packages/ai, packages/seo all built and tested locally

Day 1 afternoon (3 hours):

- Manual tasks 9–17 from §13
- Commits 6–10 from §12 (API + Dashboard + seed data)
- **Checkpoint:** You can log into dashboard at dashboard.animood.app

Day 2 morning (3 hours):

- Manual tasks 18–19
- Commits 11–17 (all frontend routes built)
- **Checkpoint:** animood.app loads, sample emotion and title pages render

Day 2 afternoon (3 hours):

- Commits 18–28 (n8n workflows + end-to-end test)
- **Final checkpoint:** Add 1 new title manually → trigger W1 through W9 → see new live page on animood.app

If you finish Day 2 with a live pipeline working for 1 title, you have succeeded. Don't worry about how many pages exist. Worry only about whether the pipeline works end-to-end. Scale comes after.

15. Phase A → Phase B Validation Criteria

Stay in Phase A (manual triggers only) until ALL of these criteria are met. Then flip to Phase B.

#	Criterion	How to test
1	Pipeline runs end-to-end without errors for 5 different titles	Run titles W1-W9 manually 5 times, all succeed
2	Extracted emotions match your own assessment in 80%+ of cases	Review 50 approved mappings, compare to your gut take
3	Evidence notes cite specific story content, not generalizations	Spot-check 20 mappings: do they reference real scenes/arcs/dialogue?
4	Confidence scores correlate with reliability	High-confidence approved at 90%+, low-confidence approved at <50%
5	Zero copyrighted text directly copied to public pages	Search 10 live pages for any 15+ word strings on Google/Reddit
6	All pages pass Google Rich Results Test	Test 10 random live pages, all valid
7	Internal linking density: 5+ links/page consistently	Spot-check 10 pages
8	Review dashboard speed: average review time < 30s per mapping	Time yourself reviewing 20 mappings

Switching to Phase B:

- In dashboard /settings, toggle from 'Phase A' to 'Phase B'
- Choose schedule: weekly (recommended) or daily
- Set OPENAI_BUDGET_USD_PER_MONTH env var as a hard cost ceiling
- Monitor for 2 weeks. If quality drops, switch back to Phase A immediately.

16. Failure Modes & Debugging Guide

Symptom	Likely Cause	Fix
W3 returns malformed JSON often	GPT-5.4 mini hitting context limit	Reduce signals per batch from 50 to 20 in W3
W2 hitting Reddit rate limits	Too many requests per minute	Add Wait node, 1s between requests
W2 hitting MAL rate limits	Default 3 req/sec	Add Wait node, 400ms between requests
Dashboard slow to load mappings	NO INDEX on status column	Add index: CREATE INDEX ON mappings(status, created_at)
Vercel build failing	Missing env vars in Vercel project	Verify all NEXT_PUBLIC_* + DATABASE_URL set in Vercel
W9 not triggering Vercel rebuild	Wrong deploy hook URL	Get new hook from Vercel project settings
pgvector queries slow	No vector index	CREATE INDEX ... USING ivfflat (embedding vector_cosine_ops)
OpenAI cost spiking	Workflow running too often	Check schedule, reduce frequency, set budget alert
Confidence scores all 'low'	Not enough signals per title	Run W2 more times before extraction
Pages have AI-fluff content	Prompt 8.3 not enforced enough	Tighten prompt rules, add more rejection criteria in W8
Schema.org validation fails	Missing required fields	Use Google Rich Results Test, fix what it complains about
Dashboard login broken	Bcrypt hash wrong format	Regenerate hash with consistent salt rounds (10)

17. Parallel SEO Learning Track

While Claude Code builds the system, you and I learn SEO architecture together. One concept per week, applied directly to AniMood. By the time the pipeline is running, you'll understand WHY each piece exists.

Week	Concept	What you'll be able to do
1	Entity-based SEO & knowledge graphs	Understand why schema.org isn't optional, how Google maps your site to
2	Topic clusters & pillar pages	Design which AniMood pages are pillars vs spokes, structure URL hierar
3	Internal linking patterns	Decide every anchor text, understand topical authority
4	Schema markup deep dive	Hand-write JSON-LD, validate it, understand which schema types Google
5	Semantic search & embeddings	Use pgvector to power 'stories like this emotionally', understand cosine sim
6	Search Console & technical SEO	Read your own data, fix crawl errors, monitor indexing
7	Content quality signals & helpful content	Understand E-E-A-T, why AI fluff gets penalized, how to add real value
8	AI Overviews & citation-worthy content	Structure content so Claude/ChatGPT/Perplexity cite your site

How the parallel track works: Each week, we have one focused conversation where I teach the concept applied to AniMood. You decide a few things based on what you learn. Claude Code implements those decisions in the next batch of commits. By Week 8, you understand the system deeply enough to maintain it without me.

This is the real value of doing the project. Even if AniMood ends up generating modest revenue, you'll have SEO + automation skills that transfer to any future site. Most builders never learn this depth because they outsource it.